UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2015-06, page [3]

Unofficial model answer:

The complete machine cycle consists of four phases: Fetch, Decode, Execute, and Store (or Write-back). A common diagram is a cycle: Fetch → Decode → Execute → Store → (back to Fetch). Each phase is described below with its function and location.

Fetch: The CPU fetches the next instruction from main memory. The Program Counter (PC) holds the address of the next instruction; this address is placed on the address bus, the instruction is read from memory via the data bus, and then the PC is incremented to point to the next instruction. The fetched instruction is placed in the Instruction Register (IR). Location: This phase occurs in the CPU (Control Unit and registers) and involves a read from main memory (RAM).

Decode: The Control Unit decodes the instruction held in the IR to determine the operation to be performed, the addressing mode, and the operands. It generates the necessary control signals for the subsequent phases. Location: This phase occurs entirely within the CPU, specifically in the Control Unit and associated decoding logic.

Execute: The CPU performs the operation specified by the instruction. This may involve an arithmetic or logic operation in the ALU, a data transfer, or a branch. Operands may be read from registers or memory. Location: This phase occurs in the CPU (ALU and registers); if memory operands are required, it also involves main memory access.

Store (Write-back): The result of the execution is written back to a register or to main memory. For a memory write, the address and data are placed on the buses and the memory is updated. Location: This phase occurs in the CPU (register write) and/or main memory (memory write).

Note: The exact diagram may vary slightly (e.g., some texts include an interrupt check or combine store with execute), but the four-phase description above is standard.

Marking points:

• *1 mark: Correctly drawing the complete machine cycle showing the four phases (Fetch, Decode, Execute, Store/Write-back) in the correct sequence.*

• *1 mark: Describing the Fetch phase: fetching instruction from memory using PC, placing in IR, incrementing PC; location: CPU and main memory.*

• *1 mark: Describing the Decode phase: control unit decodes instruction in IR to determine operation and operands; location: CPU (Control Unit).*

• *1 mark: Describing the Execute phase: ALU performs operation or data transfer; location: CPU (ALU/registers) and possibly memory.*

• *1 mark: Describing the Store/Write-back phase: writing result back to register or memory; location: CPU registers and/or main memory.*

Model confidence: high.

Model note: The question asks for a drawing; the answer describes the diagram in text. The exact phases and labels may vary by textbook, but the four-phase model is standard. No figure was provided, so the answer is based on general computer organization knowledge.

QUESTION 2 (5 marks)

Source: CSC429-2018-01, page [3]

Unofficial model answer:

a) The zero generation (pre-1940s) machines are called mechanical computers because their computation and control were performed by physical moving parts—gears, levers, wheels, and shafts—rather than by electronic circuits. They had no electronic components, no stored-program memory, and no electronic switching; operations were driven by mechanical linkages and, in some cases, by external power such as hand cranks or steam. Hence 'mechanical' describes the implementation technology of the computing hardware.

b) Table completion:
- Mechanical Calculating Machine — Inventor: Blaise Pascal (1642); Function: performs arithmetic addition and subtraction (with carry) using gear wheels; a mechanical calculator for basic arithmetic.
- Difference Machine — Inventor: Charles Babbage (1822); Function: automatically computes and tabulates polynomial functions using the method of finite differences, producing printed mathematical tables without manual arithmetic errors.

Marking points:

• 1 mark: zero generation machines used mechanical moving parts (gears/levers/wheels) for computation and control.

• 1 mark: no electronics / no stored-program concept; operation by mechanical linkages (and possibly external power).

• 1 mark: Mechanical Calculating Machine inventor = Blaise Pascal.

• 1 mark: Mechanical Calculating Machine function = mechanical arithmetic (addition/subtraction) using gears.

• 1 mark: Difference Machine inventor = Charles Babbage and function = compute/tabulate polynomials by finite differences (automated mathematical tables).

Model confidence: high.

Model note: The original table layout is not supplied; the answer assumes the two rows are 'Mechanical Calculating Machine' and 'Difference Machine' and that 'Inventor' and 'Function' are the two columns. Some syllabi may accept Leibniz for the calculating machine; Pascal's Pascaline is the standard zero-generation example. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 3 (5 marks)

Source: CSC429-2023-02, page [3]

Unofficial model answer:

The two bytes are 1101 1100 and 0010 0111. In sign-and-magnitude representation, the most significant bit (bit 7) is the sign bit (0 = positive, 1 = negative) and the remaining seven bits are the magnitude. Thus 1101 1100 represents -($101\ 1100_2$) = -(64+16+8+4) = -92, and 0010 0111 represents +($010\ 0111_2$) = +(32+4+2+1) = +39. Adding the signed values gives -92 + 39 = -53. Therefore the decimal value of the sum is -53. Workflow: (1) identify sign and magnitude bits for each byte; (2) convert each magnitude to decimal; (3) apply the sign; (4) add the two signed decimal values. Note: if the sum were to be stored back in sign-and-magnitude, -53 would be 1011 0101, but the question only asks for the decimal value of the sum.

Marking points:

• 1 mark: correctly identifies sign bit and magnitude bits for 1101 1100 (sign 1, magnitude 101 1100).

• 1 mark: correctly converts 1101 1100 to -92.

• 1 mark: correctly converts 0010 0111 to +39.

• 1 mark: correctly adds -92 + 39 = -53.

• 1 mark: clear workflow showing sign/magnitude separation and decimal conversion steps.

Model confidence: high.

Model note: Assumes standard 8-bit sign-and-magnitude with MSB as sign bit. No figure or table supplied; answer based on question text only. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION 4 (5 marks)

Source: CSC429-2026-02, page [2]

Unofficial model answer:

a) The computer interface during the third generation (mid-1960s to early 1970s) was primarily based on interactive terminals connected to mainframe computers. These terminals included teletypewriters and CRT-based dumb terminals, which allowed users to interact with the system via command-line interfaces (CLI). The interface was character-oriented and text-based, with users typing commands and receiving textual output. In the fourth generation (early 1970s to present), the interface evolved significantly with the advent of microprocessors and personal computers. This generation saw the introduction of graphical user interfaces (GUIs), starting with Xerox PARC's Alto and later commercialized by Apple Macintosh and Microsoft Windows. GUIs use windows, icons, menus, and pointing devices (WIMP) to provide a more intuitive and user-friendly interaction. Additionally, the fourth generation introduced network interfaces, allowing computers to connect to local area networks and the Internet, and later touch-based and voice-based interfaces. Overall, the third generation was characterized by text-based terminal interfaces, while the fourth generation introduced graphical, networked, and multimodal interfaces.

b) In my opinion, the leading cause that changed the computer interface from the third to the fourth generation was the invention and widespread adoption of the microprocessor. The microprocessor enabled the development of affordable personal computers, which shifted computing from centralized mainframes to individual use. This democratization of computing created a need for interfaces that were accessible to non-expert users, driving the development of graphical user interfaces. Additionally, the microprocessor's increased processing power and decreasing cost made it feasible to implement complex graphical displays and pointing devices. Furthermore, the rise of software applications for productivity, entertainment, and communication demanded more intuitive interaction methods. Thus, the microprocessor revolution, by enabling personal computing and fostering user-centric design, was the primary catalyst for the interface change.

Marking points:

• 1 mark: Third generation interface described as text-based terminal/command-line (e.g., teletype, CRT dumb terminal).

• 1 mark: Fourth generation interface described as graphical user interface (GUI) with WIMP elements (windows, icons, menus, pointer).

• 1 mark: Additional fourth-generation interface features mentioned (e.g., network interface, touch/voice, multimedia).

• 1 mark: Leading cause identified as microprocessor invention/adoption enabling personal computers.

• 1 mark: Explanation linking cause to interface change (e.g., need for user-friendly interface for non-experts, increased processing power for graphics).

Model confidence: high.

Model note: The question is open-ended; answers may vary but should cover the key interface types and a plausible cause. The marking points are indicative and not an official scheme.

QUESTION 5 (10 marks)

Source: CSC429-2025-02, page [4]

Unofficial model answer:

a) A cache is a small, fast memory that holds recently or frequently used instructions and data so that the processor can access them with much lower latency than main memory. It sits between the CPU and main memory (RAM) and exploits locality of reference (temporal and spatial) to reduce the average memory access time.

b) Without a cache, every instruction fetch and data access would have to go to main memory. Because main memory is much slower than the CPU (often tens to hundreds of cycles per access), the processor would stall frequently, drastically reducing effective throughput and increasing average memory access time. The system

© Hak Cipta Universiti Teknologi MARA

would also place heavier demand on the memory bus, causing more contention and further slowdown. Overall performance would degrade significantly, and the CPU would be underutilized.

c) L1 cache is faster than L2 cache. L1 is the first-level cache, located closest to the CPU core (often on the same die and sometimes integrated into the pipeline), and is built with very fast SRAM and a small capacity (typically a few KB to tens of KB). Its small size and proximity allow access times of a few CPU cycles. L2 is the second-level cache, larger (typically hundreds of KB to several MB) and farther from the core, so it has longer access latency (tens of cycles). The trade-off is that L1 is small and expensive per bit, while L2 is larger but slower; therefore L1 provides faster speed.

d) Two write policies for cache consistency:
1. Write-through: every write to the cache is also written immediately to main memory. This keeps main memory always up to date and ensures consistency, but it generates more memory traffic and can be slower.
2. Write-back (write-behind): writes are made only to the cache; the modified cache block is marked dirty and is written back to main memory only when it is evicted or replaced. This reduces memory traffic and is faster, but main memory can be temporarily out of date, so consistency requires careful handling (e.g., cache coherence mechanisms in multiprocessors).

Marking points:

• *1 mark: Definition of cache as small, fast memory between CPU and main memory holding recently/frequently used data/instructions.*

• *1 mark: Mention of locality of reference or reduction of average memory access time.*

• *2 marks: Explanation that without cache, every access goes to slow main memory, causing CPU stalls and much longer access times.*

• *1 mark: Additional consequence such as increased bus traffic/contention or reduced overall performance.*

• *2 marks: Identification of L1 as faster than L2.*

• *1 mark: Justification based on L1 being closer to CPU/smaller/faster SRAM (fewer cycles) while L2 is larger and farther (more cycles).*

• *1 mark: Write-through policy described (write to cache and main memory simultaneously).*

• *1 mark: Write-back policy described (write only to cache, mark dirty, write back on eviction).*

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify a particular cache organization or replacement policy, so the answer focuses on general principles. Part (d) asks for two policies to write data to the cache to ensure consistency; write-through and write-back are the standard policies, though write-back requires additional coherence mechanisms in multiprocessor systems.

QUESTION 6 (10 marks)

Source: CSC429-2024-07, page [4]

Unofficial model answer:

a) The four classifications of parallel processors (Flynn's taxonomy) are: (1) SISD — Single Instruction, Single Data: a conventional uniprocessor such as a standard PC executing one instruction stream on one data stream. (2) SIMD — Single Instruction, Multiple Data: one control unit drives many processing elements, e.g., array processors, vector processors, and GPU/SIMD extensions (MMX/SSE/AVX). (3) MISD — Multiple Instruction, Single Data: multiple instruction streams operate on the same data stream; it is rarely used in practice, with examples such as systolic arrays or fault-tolerant systems with replicated processing. (4) MIMD — Multiple Instruction, Multiple Data: multiple processors execute independent instruction streams on independent data, e.g., multicore processors, SMP servers, and clusters/distributed systems. b) One advantage of cluster computer architecture is high availability/fault tolerance: because a cluster consists of multiple independent computers connected by a network, if one node fails the remaining nodes can continue providing service (failover), so the system as a whole is more reliable than a single large machine. A second acceptable advantage is scalability/cost-effectiveness: commodity nodes can be added incrementally to

© Hak Cipta Universiti Teknologi MARA

increase performance at lower cost than a single high-end supercomputer.

Marking points:

• 1 mark: SISD correctly named with a valid example (e.g., uniprocessor/PC).

• 1 mark: SIMD correctly named with a valid example (e.g., vector/array processor, GPU, SIMD extensions).

• 1 mark: MISD correctly named with a valid example (e.g., systolic array, replicated fault-tolerant system).

• 1 mark: MIMD correctly named with a valid example (e.g., multicore, SMP, cluster).

• 2 marks: all four classifications correctly named (0.5 each) — total for part (a) = 6 marks.

• 2 marks: one clear advantage of cluster architecture stated (e.g., high availability/fault tolerance or scalability/cost-effectiveness).

• 2 marks: advantage explained with justification (e.g., node failure does not stop service; commodity nodes can be added incrementally).

• Total: 10 marks.

Model confidence: high.

Model note: Assumes the question refers to Flynn's taxonomy, the standard classification taught for parallel processors. If the course used a different taxonomy (e.g., shared-memory vs distributed-memory), the named categories would differ; the marking scheme should be adjusted accordingly. No figure or table was supplied.

QUESTION 7 (10 marks)

Source: CSC429-2018-06, page [4]

Unofficial model answer:

a) Four access methods for storage devices, each with an example: (1) Sequential access — data is read in a fixed linear order from start to end; example: magnetic tape drive. (2) Direct access — blocks are located by a block address and then read sequentially within the block; example: hard disk drive (HDD). (3) Random access — any location can be accessed in approximately the same time, independent of previous accesses; example: main memory (RAM) or solid-state drive (SSD). (4) Associative access — data is retrieved by matching content rather than by address; example: cache memory (associative cache) or content-addressable memory (CAM). (Note: 'indexed access' is sometimes listed as a fifth method, but any four of these are acceptable.)

b) With 11 address bits, the number of addressable memory locations is 2^11 = 2048 locations (2K locations).

c) For 2 MB of RAM, first express the size in bytes: 2 MB = 2 × 2^20 bytes = 2^21 bytes. If the memory is byte-addressable, the number of addressable locations is 2^21, so the required address bits = log2(2^21) = 21 bits. (If word-addressable with a different word size, the number of bits would change accordingly; the standard byte-addressable assumption gives 21 bits.)

Marking points:

• a) 1 mark for each correctly named access method with a valid example (4 marks total). No mark for a method without an example or with an incorrect example.

• b) 1 mark for the formula 2^n (or 2^11) and 1 mark for the correct result 2048 (2K) locations (2 marks total).

• c) 1 mark for converting 2 MB to 2^21 bytes (or 2,097,152 bytes); 1 mark for applying log2 to the number of locations; 1 mark for the correct answer 21 address bits (3 marks total).

• Total: 4 + 2 + 3 = 9 marks; the remaining 1 mark may be awarded for correct units/notation (e.g., 'locations', 'bits') or as a holistic accuracy mark, giving 10 marks overall.

Model confidence: high.

Model note: The question does not specify whether memory is byte-addressable or word-addressable for part (c); the standard assumption is byte-addressable, yielding 21 bits. If a word size were given, the answer would differ. Part (a) accepts any four standard access methods; 'indexed access' is also valid if used instead of one of the listed methods. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION 8 (10 marks)

Source: CSC429-2014-06, page [3]

Unofficial model answer:

a) Identification of X: X is the system bus (also called the computer interconnection network/structure). In Figure 1 it is the central interconnection that links the main memory, the processor (CPU), and the input/output peripherals, and through which communication lines to peripherals pass.

b) Function of X: The system bus provides the shared communication pathway that allows the CPU, main memory, and I/O devices to exchange data, addresses, and control signals. It carries addresses to select a memory location or device, transfers data between components, and distributes control/timing signals that coordinate transfers. Because it is shared, only one transfer (master) uses the bus at a time, so it acts as the backbone of the computer system.

c) Sub-structural components of X: The system bus is composed of three functional groups of lines: (i) the data bus, which carries the actual data between components (bidirectional, width determines transfer size); (ii) the address bus, which carries the address of the memory location or I/O port being accessed (unidirectional from master); and (iii) the control bus, which carries control and timing signals such as read/write, memory/IO select, interrupt request, clock, and bus grant/request. (Some treatments also list power/ground and arbitration lines.)

d) Importance of X: The system bus is essential because it is the single shared medium through which every component communicates; without it the CPU could not fetch instructions or data from memory nor communicate with peripherals. It determines key performance characteristics of the whole system (bus width and clock speed affect bandwidth and throughput), and it enables modularity and expandability, since devices connect to a standard bus rather than to each other directly. It is therefore the critical interconnection that makes the computer a coordinated system rather than isolated parts.

Marking points:

• *2 marks (a): correctly identifies X as the system bus / interconnection structure (1 mark for 'bus', 1 mark for 'system/computer bus' precision).*

• *3 marks (b): states it is the shared communication pathway (1 mark); mentions transfer of data/address/control between CPU, memory and I/O (1 mark); notes coordination/timing and shared single-transfer nature (1 mark).*

• *3 marks (c): data bus (1 mark); address bus (1 mark); control bus (1 mark) — accept brief function of each.*

• *2 marks (d): justified opinion on importance, e.g. enables all inter-component communication and system performance/modularity (2 marks for reasoned answer; 1 mark for a bare statement).*

Model confidence: high.

Model note: Figure 1 is supplied only by caption/extracted text; the answer assumes the standard block diagram of CPU, main memory, I/O peripherals and communication lines joined by a central interconnection labelled X. If the actual figure labels X differently (e.g. a specific bus type), the identification should be adjusted. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 9 (10 marks)

Source: CSC429-2023-07, page [3]

Unofficial model answer:

a) Two bus types available in a computer system are the address bus and the data bus. The address bus carries memory or I/O addresses from the CPU to other components, while the data bus carries the actual data between the CPU, memory, and I/O devices. (Other acceptable answers include the control bus and the system bus.)

b) System interconnection (system bus) and CPU interconnection differ in two key characteristics. First, scope and connectivity: the system bus connects the CPU, main memory, and I/O devices and is shared by all system components, whereas CPU interconnection is internal to the processor and links the CPU components

© Hak Cipta Universiti Teknologi MARA

such as registers, ALU, and control unit. Second, speed and width: the system bus is typically slower and may be narrower because it must drive external devices over longer distances, while CPU interconnection is much faster and wider to support high internal bandwidth and low latency.

c) Data transfer timing refers to how the timing of data transfers between the CPU and memory or I/O devices is coordinated. There are two main types: synchronous timing and asynchronous timing. In synchronous timing, all bus operations are synchronized to a common clock signal; data transfers occur at fixed clock intervals, which is simple and fast for devices that can operate at the bus clock rate, but it forces all devices to run at the speed of the slowest device. In asynchronous timing, there is no common clock; instead, a handshake protocol (e.g., request/acknowledge signals) is used, allowing devices of different speeds to communicate efficiently, at the cost of more complex control logic and potentially lower performance. A third type, semi-synchronous timing, is a hybrid that uses a clock but allows some handshaking for slower devices.

Marking points:

• 1 mark: Correctly lists one bus type (e.g., address bus, data bus, control bus, system bus).

• 1 mark: Correctly lists a second bus type.

• 1 mark: Identifies one valid difference between system interconnection and CPU interconnection (e.g., scope/connectivity).

• 1 mark: Provides a clear explanation of that difference.

• 1 mark: Identifies a second valid difference (e.g., speed/width).

• 1 mark: Provides a clear explanation of that second difference.

• 1 mark: Names synchronous timing and explains it (common clock, fixed intervals).

• 1 mark: Names asynchronous timing and explains it (handshake, no common clock).

• 1 mark: Contrasts the two types (e.g., simplicity vs. flexibility, slowest-device limitation vs. mixed speeds).

• 1 mark: Mentions semi-synchronous or gives a relevant example/application of each timing type.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify a figure or table, so no limitations apply. Part (a) accepts any two valid bus types; part (c) expects synchronous and asynchronous as the main types, with semi-synchronous as a possible extension.

QUESTION 10 (10 marks)

Source: CSC429-2024-07, page [4]

Unofficial model answer:

a) Three types of replacement algorithm: (1) FIFO (First-In-First-Out): the cache line that has been resident in the set the longest is chosen for replacement, maintained by a circular queue or counter; it is simple but can evict a frequently used line (Belady's anomaly possible). (2) LRU (Least Recently Used): the line that has not been referenced for the longest time is replaced, based on temporal locality; it gives good hit ratios but requires tracking recency (e.g., counters or a stack) and is costly for high associativity. (3) Random: a pseudo-randomly selected line in the set is evicted; it needs minimal hardware, avoids pathological patterns, and performs nearly as well as LRU for high associativity. (Other acceptable: LFU, NRU, optimal/OPT.)

b) Two mapping types that require a replacement algorithm: (1) Set-associative mapping (each main-memory block maps to a set of lines, so a choice within the set is needed); (2) Fully associative mapping (a block may occupy any cache line, so a replacement policy is always needed). Direct mapping does not require replacement because each block has exactly one possible line.

c) Reason for implementing a replacement algorithm: when a cache miss occurs and the target set (or the whole cache in fully associative mapping) is full, an existing valid line must be evicted to make room for the new block. The replacement algorithm decides which line to remove in order to maximise future hits by exploiting locality (e.g., keeping recently used lines). A good policy reduces the miss rate and thus the average memory access time (AMAT = hit time + miss rate × miss penalty), improving overall system performance;

© Hak Cipta Universiti Teknologi MARA

without a policy, the choice would be arbitrary and could cause excessive thrashing.

Marking points:

• 1 mark: correct description of FIFO replacement (oldest line evicted).

• 1 mark: correct description of LRU replacement (least recently used line evicted).

• 1 mark: correct description of a third algorithm such as Random (or LFU/NRU/OPT) with its basis.

• 1 mark: set-associative mapping listed as requiring replacement.

• 1 mark: fully associative mapping listed as requiring replacement.

• 1 mark: explanation that replacement is needed when a cache miss occurs and the target set/cache is full.

• 1 mark: explanation that a line must be evicted to load the new block.

• 1 mark: link to exploiting locality / keeping frequently or recently used data.

• 1 mark: link to reducing miss rate and improving average memory access time/performance.

• 1 mark: overall coherence and correct terminology (e.g., eviction, set, valid line, AMAT).

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure or table was supplied, so the answer is based solely on the question text. Part (a) asks for three types; FIFO, LRU and Random are given as the most common examples, but other valid algorithms (LFU, NRU, OPT) would also earn credit.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2026-02, page [6]

Unofficial model answer:

Interpretation of Figure 5: The expression is read as z = A + (C²/B)·D − 3A, i.e. z = A + ((C*C)/B)*D − 3*A. Registers A, B, C, D are fixed (read-only operands); x, y, z are scratch; the final result must end in z. Note: the OCR text 'z-A+c2/B*D-3A' is ambiguous, so the expression above is the assumed reading (see notes).

(a) Zero-address (stack) program. All operands must be pushed onto the stack; results are left on the stack and finally popped into z. Using the stack, compute C*C, divide by B, multiply by D, then add A and subtract 3*A:

```
PUSH C
PUSH C
MUL
PUSH B
DIV
PUSH D
MUL
PUSH A
ADD
PUSH A
PUSH A
ADD
PUSH A
ADD
SUB
POP z
```

(Equivalently, the 3A term can be formed as PUSH A / PUSH A / ADD / PUSH A / ADD, then SUB.) The stack holds the running value; the last POP z stores the final answer in z.

(b) One-address program. One operand is the accumulator (implicit) and the other is a named register; results go to the accumulator. Use x and y as temporaries:

```
LOAD C
MUL C
STOR x
LOAD x
DIV B
STOR x
LOAD x
MUL D
STOR x
LOAD A
ADD x
STOR y
LOAD A
MUL A
ADD A
ADD A
```

© Hak Cipta Universiti Teknologi MARA

STOR x
LOAD y
SUB x
STOR z

Here x holds the C²/B·D term, y holds A + that term, and x is reused to hold 3A; the final SUB leaves the answer in the accumulator, which is stored to z.

(c) Three-address program. Each instruction names two sources and one destination, so the expression is built directly with temporaries x and y:

MUL (C, C, x)
DIV (x, B, x)
MUL (x, D, x)
ADD (A, x, y)
MUL (A, A, x)
ADD (x, A, x)
ADD (x, A, x)
SUB (y, x, z)

This computes x = C²/B·D, y = A + x, x = 3A, and finally z = y − x = A + C²/B·D − 3A, with the answer in z as required.

Marking points:

• *2 marks: correct interpretation of Figure 5 as z = A + (C*C)/B*D - 3*A (or clearly stated equivalent)*

• *6 marks: zero-address program — correct PUSH/POP usage, correct operator order (MUL, DIV, MUL, ADD, SUB), final POP z (2 marks per logical stage)*

• *6 marks: one-address program — correct LOAD/STOR with accumulator, correct use of temporaries x and y, correct final STOR z*

• *4 marks: three-address program — correct ADD/SUB/MUL/DIV (p,q,r) form, correct temporaries, final result in z*

• *2 marks: overall correctness — final answer stored in z in all three versions and fixed registers A, B, C, D never overwritten*

Model confidence: medium.

*Model note: Figure 5 is supplied only as OCR text 'z-A+c2/B*D-3A'; the expression was interpreted as z = A + C^2/B*D - 3A. If the intended expression differs (e.g. z = A + C^2/(B*D) - 3A or z = A + C^2/B*D - 3*A with different grouping), the instruction sequences must be re-ordered accordingly. Table 1 gives only the instruction mnemonics; operand conventions (e.g. whether one-address ADD p means ACC <- ACC + p) are assumed standard. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA